



Introducción a JavaScript

Apunte de herramientas

Tecnicatura Superior de Desarrollo de Software - Profesor Alexander Seling

Instituto Superior de Formación Técnica Leopoldo Marechal, Ituzaingó.

El objetivo en esta materia no es aprender desarrollo web, sino poder desarrollar un Producto Mínimo Viable a partir de un caso de estudio.

Ya hicimos el análisis, ya hicimos el maquetado de una interfaz web.

Ahora comenzamos a incorporar funcionalidades y a trabajar con datos.

1. ¿Qué se puede hacer con JavaScript?

- Modificar, mostrar u ocultar elementos HTML.
- Modificar estilos y clases CSS.
- Responder a acciones del usuario, como clics, formularios o teclas.
- Validar y procesar información ingresada por el usuario.
- Trabajar con datos mediante variables, arrays y objetos.
- Consultar servidores y recibir información desde una API.
- Enviar información a un servidor.
- Realizar operaciones asíncronas sin bloquear la interfaz.

2 ¿Dónde escribimos JavaScript?

De forma similar a CSS, podemos incorporar JavaScript de distintas maneras dentro de un documento HTML.

JavaScript externo

El código se encuentra en un archivo `.js` separado.

```
<script src="js/script.js" >  
</script>
```

JavaScript inline

El código se escribe directamente dentro de un elemento HTML:

```
<button onclick="saludar()" >  
  Saludar  
</button>
```

JavaScript interno

El código se escribe dentro del propio documento HTML, utilizando `<script>`:

```
<script>  
  console.log("Hola");  
</script>
```

2.1. ¿Cuál vamos a utilizar?

Al igual que hicimos con CSS, en nuestro proyecto vamos a trabajar principalmente con **archivos externos**.

Esto permite mantener separado:

HTML:

Estructura

CSS:

Presentación

JavaScript:

Comportamiento

Por ejemplo, nuestra estructura de proyecto puede ser:

```
portal-paciente/  
├── index.html  
├── css/  
│   └── estilos.css  
└── js/  
    └── script.js
```

3. Variables y constantes

JavaScript permite declarar variables utilizando principalmente `let` y `const`.

```
let edad = 35;  
const nombre = "María";
```

La diferencia principal es que una variable declarada con `const` no puede ser reasignada:

```
edad = 36; // reasignación, correcto  
nombre = "Ana"; // Error, ya fue declarado constante como María.
```

📌 **Como criterio general utilizaremos `const` por defecto y `let` cuando necesitemos reasignar el valor de una variable**

⚠️ JavaScript también permite utilizar `var`, una forma de declaración más antigua que no utilizaremos en este proyecto.

4. Operadores

JavaScript utiliza operadores similares a los que ya conocemos de otros lenguajes como C.

Operadores lógicos

```
const and = true && false // false
const or = true || false // true
const not = !true // false
```

Operadores aritméticos

```
const veinte = 15 + 5;
const ocho = 4 * 2;
const uno = 9 % 2;
const siete = 14 / 2
```

Operadores de asignación

```
= += -= *= /=
```

```
let contador = 0;
contador += 1;
```

Operadores de comparación

Mayor, menor o igual:

```
4 > 7 // false
12 <= 12 // true
```

Igualdad de valor

```
"123" == 123 // true
"123" != 123 // false
```

Igualdad de valor y de tipo:

```
"123" === 123 // false
"123" !== 123 // true
```

5. Tipos de datos

Algunos de los tipos de datos más utilizados en JavaScript son:

```
const nombre = "María";    // string
const edad = 35;           // number
const activo = true;       // boolean
const turno = null;        // ausencia intencional de valor
let resultado;             // undefined
```

 También podemos trabajar con estructuras de datos que contengan cualquier tipo de datos.

6. Funciones en JavaScript

Función tradicional

```
function sumar(a, b) {  
  return a + b;  
}
```

Función anónima

```
const sumar = function(a, b) {  
  return a + b;  
};
```

Función flecha

```
const sumar = (a, b) => {  
  return a + b;  
};
```

 A diferencia de C, JavaScript **no requiere declarar el tipo de los parámetros ni el tipo de retorno.**

Podemos llamar a la función de la misma manera:

```
const resultado = sumar(10, 20);
```

6.1 Funciones como parámetro de funciones

En JavaScript podemos almacenar una función en una variable:

```
const saludar = () => {  
  console.log("Hola");  
};
```

También podemos definir una función como argumento de otra función:

```
setTimeout(() => {  
  console.log("Hola");  
}, 1000);
```

Esta característica será importante cuando trabajemos con métodos de arrays y eventos.

Por ejemplo, `forEach()` recibe una función que ejecutará para cada elemento del array.

7. Estructuras de datos

Arrays

También conocidos como **arreglos**, **listas** o **vectores**

```
const especialidades = [  
  "Clínica médica",  
  "Cardiología",  
  "Oftalmología"  
];
```

Objects

También conocidos como objetos, mapas o diccionarios

```
const paciente = {  
  nombre: "María",  
  edad: 35,  
  obraSocial: "IOMA"  
};
```

❏ Los arrays y objetos serán especialmente importantes para trabajar con datos provenientes de archivos JSON.

8. Objetos: claves y valores

Los objetos permiten representar información agrupando diferentes datos relacionados.

```
const paciente = {  
  nombre: "María González",  
  edad: 35,  
  obraSocial: "Salud Total"  
};
```

Cada propiedad está formada por una **clave** y un **valor**:

```
nombre → "María González"  
edad → 35  
obraSocial → "Salud Total"
```

Podemos acceder a una propiedad utilizando el operador `.`:

```
console.log(paciente.nombre);
```

También podemos utilizar la notación con corchetes:

```
console.log(paciente["nombre"]);
```

Ésta forma resulta especialmente útil cuando el nombre de la propiedad se encuentra almacenado en una variable.

9. Arrays de objetos

En una aplicación es habitual trabajar con una colección de elementos.

Por ejemplo, los turnos de un paciente:

```
const turnos = [  
  {  
    especialidad: "Cardiología",  
    fecha: "10/09/2026",  
    hora: "10:30"  
  },  
  {  
    especialidad: "Clínica médica",  
    fecha: "15/09/2026",  
    hora: "14:00"  
  }  
];
```

Podemos acceder a un elemento del array:

```
console.log(turnos[0]);
```

Y a una propiedad de ese elemento:

```
console.log(turnos[0].especialidad);
```

 Este tipo de estructura será muy frecuente cuando trabajemos con información proveniente de archivos JSON.

10. Recorrer arrays

El método `forEach()` permite ejecutar una función para cada elemento de un array.

```
turnos.forEach(function(turno) {  
  console.log(turno.especialidad);  
});
```

También podemos utilizar una función flecha:

```
turnos.forEach(turno => {  
  console.log(turno.especialidad);  
});
```

En ambos casos, la función recibe como parámetro cada elemento del array.

Esto será fundamental para transformar una colección de datos en elementos HTML.

11. Desestructuración

JavaScript permite extraer propiedades de un objeto directamente en variables.

```
const paciente = {  
  nombre: "María",  
  edad: 35  
};  
const { nombre, edad } = paciente; //declara las variables nombre y edad con los valores María y 35
```

Ahora podemos utilizar directamente:

```
console.log(nombre);  
console.log(edad);
```

También podemos desestructurar arrays:

```
const especialidades = [  
  "Clínica médica",  
  "Cardiología"  
];  
const [primera, segunda] = especialidades;
```

La desestructuración es frecuente al trabajar con datos y con las APIs de JavaScript.

12. Eventos

Los eventos representan acciones o situaciones que ocurren en el navegador.

Algunos ejemplos:

- `click`: el usuario hace clic.
- `submit`: se envía un formulario.
- `input`: cambia el contenido de un campo.
- `change`: cambia el valor de un control.
- `keydown`: se presiona una tecla.

JavaScript permite asociar una función a un evento mediante `addEventListener()`.

```
const boton = document.querySelector("#btn-turno");  
boton.addEventListener("click", () => {  
  console.log("Botón presionado");  
});
```

La función se ejecutará cada vez que ocurra el evento indicado.

13. Eventos de formularios

Podemos detectar el envío de un formulario utilizando el evento `submit`.

```
const formulario = document.querySelector("#form-login");

formulario.addEventListener("submit", event => {
  event.preventDefault();
  console.log("Formulario enviado");
});
```

- ❏ En un formulario, el comportamiento habitual es enviar los datos y navegar a otra página.
`preventDefault()` permite cancelar ese comportamiento para que JavaScript pueda procesar el evento.

⚠ Esto será útil para validar y procesar formularios desde JavaScript, **antes de contactar con nuestro backend.**

14. Manipulación del DOM

El **DOM (Document Object Model)** es la representación del documento HTML que construye el navegador.

JavaScript puede utilizar el DOM para buscar, modificar, crear y eliminar elementos.

Por ejemplo:

```
<p id="mensaje">Sin turnos disponibles.</p>
```

Podemos buscar ese elemento:

```
const mensaje = document.querySelector("#mensaje");
```

Y modificar su contenido:

```
mensaje.textContent = "Tiene 2 turnos disponibles.";
```

Algunas herramientas que utilizaremos:

```
document.querySelector()  
element.textContent  
element.value  
element.classList  
document.createElement()  
element.appendChild()
```

15. Crear elementos dinámicamente

JavaScript puede crear nuevos elementos HTML utilizando `createElement()`.

```
const elemento = document.createElement("li");  
  
elemento.textContent = "Cardiología - 10/09/2026";
```

Después podemos incorporarlo al documento:

```
const lista = document.querySelector("#lista-turnos");  
  
lista.appendChild(elemento);
```

De esta forma podemos construir una interfaz a partir de datos que se encuentran en JavaScript.

16. JSON

JSON (JavaScript Object Notation) es un formato de texto utilizado para representar e intercambiar datos.

Su estructura está basada en objetos y arrays.

Por ejemplo:

```
{  
  "nombre": "María González",  
  "edad": 35,  
  "rol": "paciente"  
}
```

También podemos representar una colección:

```
[  
  {  
    "especialidad": "Cardiología",  
    "fecha": "10/09/2026"  
  },  
  {  
    "especialidad": "Clínica médica",  
    "fecha": "15/09/2026"  
  }  
]
```

JSON no es JavaScript, aunque su sintaxis resulta muy similar a la utilizada para representar objetos y arrays en JavaScript.

16.1. JSON y JavaScript

Un archivo JSON contiene **texto con una estructura determinada**.

Cuando JavaScript recibe ese contenido, podemos convertirlo en valores que el programa pueda utilizar.

Por ejemplo, este archivo:

```
{  
  "nombre": "María",  
  "edad": 35  
}
```

puede convertirse en un objeto JavaScript:

```
const paciente = {  
  nombre: "María",  
  edad: 35  
};
```

Esta conversión será realizada automáticamente mediante los métodos que veremos al utilizar `fetch()`.

17. Fetch API

La API `fetch()` permite realizar solicitudes de recursos utilizando JavaScript.

Podemos utilizarla para obtener:

- archivos JSON;
- información desde una API;
- otros recursos disponibles mediante HTTP.

En nuestro proyecto comenzaremos utilizando un archivo JSON local:

```
data/  
└── data.json
```

Podemos solicitarlo mediante:

```
fetch("data/data.json");
```

`fetch()` realiza una operación asíncrona: el programa inicia la solicitud y el resultado estará disponible posteriormente.

17.1. `async` y `await`

Una forma sencilla de trabajar con el resultado de `fetch()` es utilizar `async` y `await`.

```
async function cargarDatos() {  
  const response = await fetch("data/data.json");  
  const data = await response.json();  
  console.log(data);  
}
```

¿Qué ocurre?

```
fetch()  
↓  
respuesta HTTP  
↓  
response.json()  
↓  
datos JavaScript
```

`await` permite esperar el resultado de una operación asíncrona antes de continuar con la siguiente instrucción.

La función que utiliza `await` debe estar declarada como `async`.

18. Del JSON al HTML

Una vez obtenidos los datos, podemos recorrerlos y construir elementos HTML dinámicamente.

Supongamos que `data.json` contiene:

```
[
  {
    "especialidad": "Cardiología",
    "fecha": "10/09/2026"
  },
  {
    "especialidad": "Clínica médica",
    "fecha": "15/09/2026"
  }
]
```

Podemos cargar y renderizar esos datos:

```
async function cargarTurnos() {
  const response = await fetch("data/data.json");
  const turnos = await response.json();
  const lista = document.querySelector("#lista-turnos");
  turnos.forEach(turno => {
    const elemento = document.createElement("li");
    elemento.textContent = `${turno.especialidad} - ${turno.fecha}`;
    lista.appendChild(elemento);
  });
}
```

El proceso completo es:

```
data.json
↓
fetch()
↓
response.json()
↓
array de objetos
↓
forEach()
↓
createElement()
↓
appendChild()
↓
HTML
```

Los datos dejan de estar escritos directamente en el HTML y pasan a utilizarse para construir dinámicamente la interfaz.

19. ¿Qué estamos construyendo?

Hasta ahora nuestro HTML contenía directamente la información que mostraba la página.

Ahora podemos separar:

```
ESTRUCTURA
HTML
+
PRESENTACIÓN
CSS
+
DATOS
JSON
+
LÓGICA
JavaScript
```

JavaScript se encarga de **obtener los datos, procesarlos y construir o modificar la interfaz**.

En nuestro proyecto, esta separación nos permitirá avanzar desde un prototipo estático hacia una aplicación basada en datos.

Más adelante reemplazaremos el archivo JSON local por información obtenida desde un servidor y una API.